Dokumentacja Projektowa

Graph Layout Visualizer — JIMP2 Project

Mikołaj Nierodziński Nghia Bao Phuc Ho

April 2026

Contents

1 Project Overview

This document serves as the design-level documentation (*dokumentacja projektowa*) for the JIMP2 graph layout project. It complements the functional specification (week 2) and implementation documentation (week 4) by providing the overall system architecture, module interactions, testing strategy, and evaluation of results.

The program implements two graph layout algorithms in C — Fruchterman-Reingold (force-directed) and Tutte's embedding — and outputs vertex coordinates that can be visualized using an external Python script or consumed by a downstream Java application.

1.1 Intended Audience

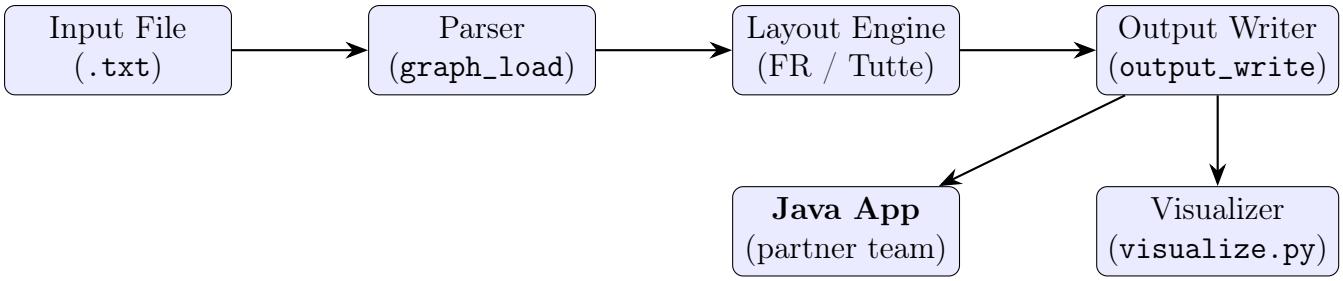
This documentation is written for:

- The course instructors evaluating the project.
- A partner team developing a **Java-based visualization application** that reads the output produced by this C program. The Java team should refer to Section 4 for the exact output format specification and Section 5 for integration guidelines.

2 System Architecture

2.1 High-Level Pipeline

The program follows a linear pipeline architecture:



Note that the visualizer in python is only for testing purposes, not the final project's goal

2.2 Module Decomposition

The source code is organized into the following modules:

File	Responsibility	Key Functions
main.c	Program entry point, argument parsing	main
graph.c / graph.h	Graph loading, validation, memory	graph_load, graph_free
vertex.h	Vertex data structure definition	—
edge.h	Edge data structure definition	—
layout_fruchterman_reingold.c	Force-directed layout	layout_fruchterman_reingold
layout_tutte.c	Tutte's convex embedding	layout_tutte
output.c / output.h	Text and binary output	output_write
adjacency.c / adjacency.h	Adjacency list construction	build_adjacency
visualize.py	Matplotlib-based PNG generation	—

Table 1: Module overview

2.3 Directory Structure

```
project/
|-- src/
|   |-- main.c
|   |-- graph.c / graph.h
|   |-- vertex.h
|   |-- edge.h
|   |-- layout_fruchterman_reingold.c / .h
|   |-- layout_tutte.c / .h
|   |-- output.c / output.h
|-- bin/
|   |-- graph              (compiled binary)
|-- visualize.py
|-- Makefile
|-- Dockerfile
|-- README.md
```

3 Data Flow

3.1 Internal Data Lifecycle

1. **Loading:** `graph_load` reads the input file in two passes. Pass 1 validates all lines and counts edges/vertices. Pass 2 fills the allocated arrays and checks for duplicates.

2. **Connectivity check:** A BFS traversal from vertex 0 verifies that the graph is connected.

3. **Layout computation:** The selected algorithm modifies the `x` and `y` fields of each `vertex_t` in place.

4. **Output:** The final coordinates are written to a file in text or binary format.

5. **Cleanup:** `graph_free` deallocates all heap memory in reverse order.

3.2 Memory Management Summary

All dynamic memory follows a strict allocate-in-load, free-in-cleanup discipline. No global state is used. The layout functions allocate only temporary buffers (displacement arrays for FR, adjacency lists for Tutte), which are freed before the function returns.

4 Output Format Specification

This section provides the definitive specification for teams consuming the program's output.

4.1 Text Format (`text`)

Each line contains one vertex record:

```
<id> <x> <y>
```

- `id` — integer vertex identifier (same as in the input file).
- `x`, `y` — floating-point coordinates formatted with `%g`.
- Fields are separated by a single space.
- Vertices are ordered by ascending ID.
- No header line is present.

Example:

```
1 0.5 0.866025
2 -0.5 0.866025
3 -1 0
4 0.12 -0.34
```

4.2 Binary Format (`bin`)

Each vertex is stored as a packed 20-byte record:

Field	C Type	Size	Byte Offset
`id`	`int`	4 bytes	0
`x`	`double`	8 bytes	4
`y`	`double`	8 bytes	12

Records are sequential, no padding, native little-endian (x86/x86-64). Total file size: $n \times 20$ bytes, where n is the number of vertices.

5 Integration Guide for the Java Team

This section is intended for the partner team that will build a Java visualization application consuming the output of this C program.

5.1 Workflow

1. Prepare an input graph file following the edge-list format (see Section 2.1 of the functional specification).
2. Run the C program:

   ```
   ./bin/graph input.txt output.txt fr text
   ```

3. Parse the resulting `output.txt` in Java to extract vertex coordinates.
4. To draw edges, the Java application must also read the **original input file** to obtain the edge list (vertex pairs and names).

5.2 Parsing the Text Output in Java

A minimal Java reader for the text format:

```
    BufferedReader reader = new BufferedReader(new FileReader("
       output.txt"));
    String line;
    while ((line = reader.readLine()) != null) {
        String[] parts = line.trim().split("\\s+");
        int id    = Integer.parseInt(parts[0]);
        double x  = Double.parseDouble(parts[1]);
        double y  = Double.parseDouble(parts[2]);
        // store (id, x, y)
    }
    reader.close();
```

5.3 Parsing the Binary Output in Java

```
    DataInputStream dis = new DataInputStream(new FileInputStream("
       output.bin"));
    // Note: C writes native little-endian;
    // Java's DataInputStream reads big-endian.
    // Use ByteBuffer for correct byte order:
    byte[] buf = new byte[20];
    while (dis.read(buf) == 20) {
        ByteBuffer bb = ByteBuffer.wrap(buf)
            .order(ByteOrder.LITTLE_ENDIAN);
        int id    = bb.getInt();
        double x  = bb.getDouble();
        double y  = bb.getDouble();
        // store (id, x, y)
    }
    dis.close();
```

5.4 Coordinate System

- **Fruchterman-Reingold:** Coordinates range from 0 to 800 (x) and 0 to 600 (y), corresponding to the viewport dimensions defined in the C code. The Java application can map these directly to a canvas or scale them.
- **Tutte:** Coordinates are centered around the unit circle (-1 to $+1$ on both axes). The Java application should scale and translate these to fit the display area.

5.5 Exit Codes

If the C program terminates with a non-zero exit code, the output file may be absent or incomplete. The Java application should check the process exit code before attempting to parse the output. Refer to Section 5 of the functional specification for the full list of exit codes and their meanings.

6 Testing Strategy

6.1 Test Cases

Test ID	Category	Description
T01	Basic	Triangle graph (3 vertices, 3 edges)
T02	Basic	Complete graph K_5 (5 vertices, 10 edges)
T03	Medium	Grid graph 4×4 (16 vertices)
T04	Large	Random connected graph with 100+ vertices
T05	Edge case	Graph with a single edge (2 vertices)
T06	Error	Empty file — expect exit code 5
T07	Error	Self-loop — expect exit code 8
T08	Error	Duplicate edge — expect exit code 10
T09	Error	Disconnected graph — expect exit code 11
T10	Error	Negative weight — expect exit code 9
T11	Error	Invalid ID (zero or negative) — expect exit code 7
T12	Format	Binary output, verified by size ($n \times 20$ bytes)
T13	Algorithm	Compare FR and Tutte on the same graph

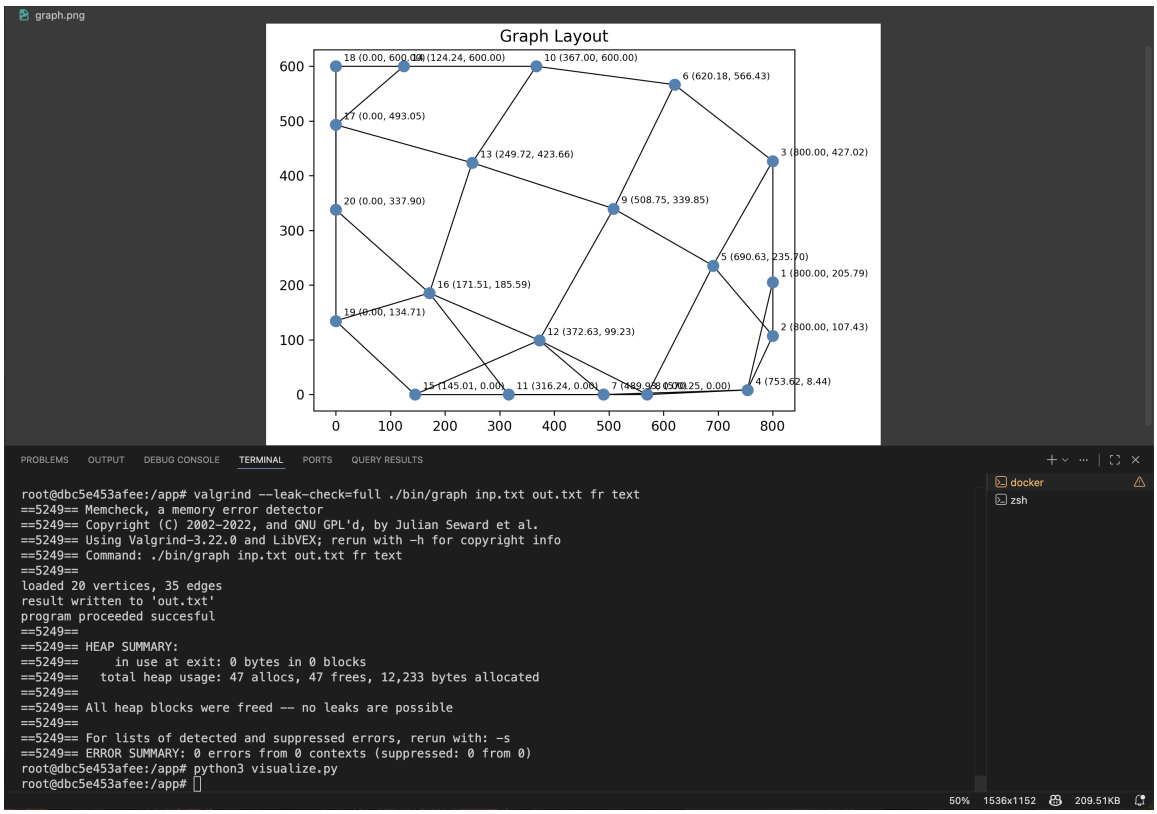
Table 2: Test case summary

6.2 Memory Leak Verification

All test cases were run under Valgrind:

```
valgrind --leak-check=full --error-exitcode=1
./bin/graph input.txt output.txt fr text
```

6.3 Example Test Results



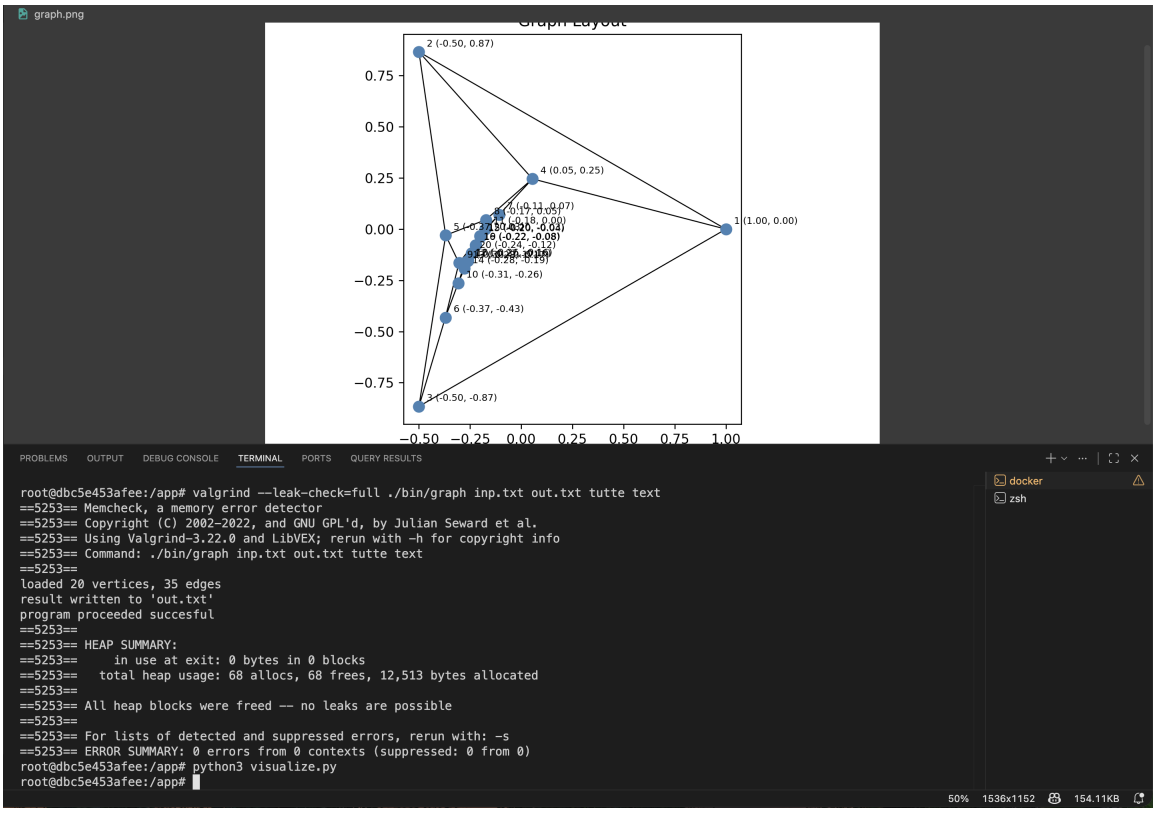


Figure 1: Fruchterman-Reingold (top) vs Tutte (bottom) for the same input

7 Algorithm Comparison

Aspect	**Fruchterman-Reingold**	**Tutte's Embedding**
Type	Force-directed (iterative)	Linear system (Gauss-Seidel)
Deterministic	No (random initial positions)	Yes (fixed outer face)
Guarantees planarity	No	Yes (for 3-connected planar)
Edge weights	Yes (scales attractive force)	No
Coordinate range	$[0, 800] \times [0, 600]$	$\approx [-1, 1] \times [-1, 1]$
Best suited for	General graphs, visual appeal	Planar graphs, crossing-free
Time complexity	$O(I \cdot (n^2 + m))$	$O(I \cdot (n + m))$

Table 3: Comparison of the two layout algorithms. I = iterations, n = vertices, m = edges.

8 Division of Work

Team Member	**Responsibilities**
Mikołaj Nierodziński	Repository setup and management. Implementation of the Fruchterman-Reingold algorithm. Documentation preparation and editing.
Nghia Bao Phuc Ho	Implementation of Tutte's embedding algorithm. Graph loading, validation, and error handling. Binary and text output module. Testing and memory leak verification with Valgrind.

Table 4: Division of work

9 Challenges and Solutions

1. **Division by zero in FR algorithm:** When two vertices share the exact same coordinates, the distance becomes zero, causing a division-by-zero error in the repulsive force calculation. This was solved by assigning random initial positions to all vertices.

2. **Explosive movement in early iterations:** Without temperature capping, vertices would overshoot their optimal positions. The simulated annealing approach (decreasing temperature) resolved this.

3. **Duplicate edge detection performance:** The current linear scan for duplicate detection has $O(m^2)$ worst-case complexity. For the graph sizes tested (up to a few hundred edges), this was acceptable, but a hash-based approach would be needed for larger inputs.

10 Conclusion

The project successfully implements two distinct graph layout algorithms in C, providing both text and binary output formats. The Fruchterman-Reingold algorithm produces visually appealing layouts for general graphs, while Tutte's embedding guarantees crossing-free drawings for 3-connected planar graphs. The output format is designed to be easily consumed by external tools, including the partner team's Java-based visualization application.